More practice - Tries

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	1
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Tries | 5 questions | tap a title to jump to it

M	1. Solve chat, grid paths, and car rentals	PracHub	2025-12-02
M	2. Greedy Longest-Match Dictionary Tokenizer	1Point3Acres	2026-06-15
M	3. Check if all substrings are dictionary words	PracHub	2026-03-11
M	4. Return dictionary words matching a prefix	PracHub	2026-03-09
H	5. Implement Longest-Match Text Replacement	PracHub	2026-01-20

STUDY PATH: question 148 of 167 (more practice)

Priority score 3.32 | Tries | asked 322 days ago

MEDIUM | PracHub | Technical Screen | 1 of 5

★ TOP 75: rank 70 in the Medium and Hard priority list

★ Solve chat, grid paths, and car rentals

POSTED 2025-12-02 | 322 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 43/228

ABOUT THIS QUESTION

Topic: More practice - Tries

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words, LeetCode #62 Unique Paths, LeetCode #253 Meeting Rooms II

LeetCode match: LC 692 Top K Frequent Words + LC 62/63 Unique Paths + LC 253

Posted on PracHub: 2025-12-02 (updated 2026-05-02)

Original: https://prachub.com/interview-questions/solve-chat-grid-paths-and-car-rentals

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.

Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **`k`** (1 <= k <= number of distinct users).

Output:

- Return a list of **`k`** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **`k`** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **`k`** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:

QUESTION

- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

QUESTION

Tasks: 1.
Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.
Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.
Output:

- An integer `C` = minimal number of cars needed.
- A mapping from each `rental_id` to a `car_id` in the range `1..C`, such that no two rentals assigned to the same `car_id` overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals `[s1, e1)` and `[s2, e2)` overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if `e1 == s2` they do **not** over

APPROACH

Top-K messages via counts + heap; grid paths via DP (with obstacles if present); car rentals = minimum fleet = max overlapping reservations (sweep line).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Three independent tasks. Chat: activity means the number of messages, ties resolved how? Grid paths: moves right or down only, with or without obstacles, modulo? Car rentals: minimum cars equals the maximum overlap of reservations; does a car returned at time t serve a rental starting at t?

BRUTE FORCE TO SAY OUT LOUD

Chat: sort the counts. Grid: recursion without memory. Cars: assign reservations to cars one by one.

OPTIMAL IDEA

Chat: Counter and heapq.nsmallest with the key (count descending, user id ascending). Grid: dp over cells with a rolling row, dp[j] += dp[j - 1], and 0 on obstacles; O(mn) time, O(n) space. Cars: sort the reservations by start and keep a min-heap of end times; reuse the earliest-ending car if it is free, else add one. The heap size at the end is the answer.

DRY-RUN THESE CASES

Chat: ties, fewer users than K. Grid: 1 x n, an obstacle on the start, no path. Cars: back-to-back rentals, one rental, all identical.

LIKELY FOLLOW-UPS

Chat in a stream, grid paths with diagonal moves, cars with a fixed fleet size and rejecting requests.

PITFALLS

Treating back-to-back as overlapping. Forgetting the modulo in the grid count if the statement requires it.

HOW TO THINK ABOUT IT

Three unrelated tasks, so say so and give each a one-line plan. Chat: count, then take the top K with a heap. Grid: each column depends only on the previous one, so keep one column. Cars: the minimum number of cars is the largest number of rentals that overlap at once, which a heap of return times tracks while also giving an assignment.

Basis: Three independent tasks: top K frequent items (LeetCode 347 / 692), grid path counting (a Unique Paths variant) and minimum meeting rooms with assignment (LeetCode 253 plus the assignment step).

BRUTE FORCE · Q1: sort all users by (count, user id)

BRUTE FORCE · PSEUDO-CODE

```
def top_k_users(log, k):
    counts = Counter(msg.user for msg in log)
    def rank(u):
        return (-counts[u], u)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting all d distinct users costs O(d log d) even though only K are needed; a heap of size K costs O(d log k).

BRUTE FORCE · Q2: plain recursion over the moves

BRUTE FORCE · PSEUDO-CODE

```
def ways(r, c):
    if c == n - 1:
        return 1 if r == n - 1 else 0
    total = 0
    for dr in (-1, 0, 1):
        if 0 <= r + dr < n:
            total += ways(r + dr, c + 1)
    return total
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three calls per cell and the same cell is reached by many paths, so the call tree has about 3^n nodes. One column at a time with a rolling array is O(m * n).

BRUTE FORCE · Q3: count the overlap at every time and assign greedily without a heap

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE · Q3: count the overlap at every time and assign greedily without a heap

```
def cars_needed(rentals):
    free_at = []  # per car
    for start, end in sorted(rentals):
        for i, f in enumerate(free_at):
            if f <= start:
                free_at[i] = end  # reuse
                break
        else:
            free_at.append(end)  # new car
    return len(free_at)
```

TIME	SPACE
O(R^2)	O(R)

WHY IT IS SLOW

Each rental scans every car, so the cost is O(R * cars), up to O(R^2). A heap of free times finds the earliest-free car in O(log R).

OPTIMAL SOLUTION · Q1: top-K active users with a size-K heap

TIME	SPACE
O(n + d log k) for d distinct users	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: count constrained paths with a rolling column

TIME	SPACE
O(m * n)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q3: minimum cars and one valid assignment

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1: what if the log is a stream and memory is limited?

Use a heavy-hitters sketch: Misra-Gries keeps k counters and is exact for any user with more than n / (k + 1) messages; a count-min sketch plus a small heap estimates the top K in O(1) memory per update. For exact answers across machines, count per shard, send each shard's local top-K plus a threshold, and merge.

FOLLOW-UP 2 Q2: what changes if there are obstacles or the path may also go straight up and down?

Obstacles just force ways[r] = 0 for blocked cells in the column being built. Allowing vertical moves inside a column makes each column a chain, so the transition becomes a linear recurrence solved by a prefix pass, still O(m * n). For huge n, use matrix exponentiation of the m x m transition matrix: O(m^3 log n).

FOLLOW-UP 3 Q3: how do you answer 'is one more rental possible' online?

Keep the busy heap and the free-car count between calls. A new rental fits without a new car if any car's return time is <= its pickup, which is a heap peek after popping everything that returned before the pickup. That is O(log R) per query.

FOLLOW-UP 4 Q2 with obstacles.

A blocked cell contributes zero ways when its column is built; everything else is unchanged. O(m * n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve chat, grid paths, and car rentals
#
# You are given three independent programming tasks. For
# each, describe and implement an efficient algorithm, and
# be prepared to analyze its time and space complexity.
# --- ### Q1. Find top-K most active chat users You are
# given a log of chat messages from a messaging
# application. **Input**: - An array messages of length n,
# where each element messages[i] is the user_id (string or
# integer) of the sender of the i-th message. - An integer
# k (1 <= k <= number of distinct users). **Output**: -
# Return a list of k user IDs corresponding to the users
# who have sent the most messages. **Details &
# requirements**: - The "activity" of a user is defined as
# the number of messages they have sent (i.e., their
# frequency in messages). - If there are fewer than k
# distinct users, return all distinct users.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1: what if the log is a stream and memory is
# limited?
# 2. Q2: what changes if there are obstacles or the path
# may also go straight up and down?
# 3. Q3: how do you answer 'is one more rental possible'
# online?
# 4. Q2 with obstacles.
#
# INTERVIEW NOTES
# Three unrelated tasks, so say so and give each a one-
# line plan. Chat: count, then take the top K with a heap.
# Grid: each column depends only on the previous one, so
# keep one column. Cars: the minimum number of cars is the
# largest number of rentals that overlap at once, which a
# heap of return times tracks while also giving an
# assignment.
#
```

```
# BRUTE FORCE (say it first): Q1: sort all users by
# (count, user id)
# Count the messages and sort all distinct users, then
# take the first K. Correct and the oracle; sorting all d
# users does more work than a heap of size K. Time O(d log
# d). Space O(d).


import heapq
```

```
from collections import Counter


# BRUTE FORCE (say it first): Q1: sort all users by
# (count, user id) | time O(d log d)
def top_users_sort(messages, k):
    c = Counter(messages)
    return sorted(c, key=lambda u: (-c[u], u))[:k]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def top_k_users(messages, k):
    # Q1, heap: O(n + d log k); sorted order breaks ties
    counts = Counter(messages)
    best = heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))
    return [user for user, _ in best]


# Q1 second approach: sort every user
def top_k_users_sorted(messages, k):
    # Q1, second approach: sort every user, O(d log d)
    counts = Counter(messages)
    ranked = sorted(counts, key=lambda u: (-counts[u], u))
    return ranked[:k]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def count_paths(m, n, blocked=(), mod=None):
    # Q2: start (m-1, 0), end (m-1, n-1), moves right-up,
    # right, right-down. One column of ways is enough:
    # O(m).
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
```

```
    for c in range(1, n):
        nxt = [0] * m
        for r in range(m):
            # FOLLOW-UP 4: blocked cells (obstacles) give
            # no paths
            if (r, c) in wall:
                # a blocked cell contributes no paths
                continue
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            nxt[r] = total % mod if mod else total
        ways = nxt
    return ways[m - 1]
```

```
# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def min_cars(rentals):
    # Q3: rentals are (pickup, return); how many cars are
    # needed?
    # A heap holds the return times of the cars on the
    # road.
    cars = []
    for pickup, back in sorted(rentals):
        if cars and cars[0] <= pickup:
            # a car came back in time: reuse it
            heapq.heapreplace(cars, back)
        else:
            heapq.heappush(cars, back)
    return len(cars)


# FOLLOW-UP 3: Q3: how do you answer 'is one more rental
# possible' online?
# Answer: Keep the busy heap and the free-car count
# between calls. A new rental fits without a new car if
# any car's return time is <= its pickup, which is a heap
# peek after popping everything that returned before the
```

```
# pickup.
class RentalDesk:
    """Online: can one more rental be served right now?"""

    def __init__(self, fleet):
        self.fleet, self.out = fleet, []

    def rent(self, pickup, back):
        # cars returned by `pickup` are free again
        while self.out and self.out[0] <= pickup:
            heapq.heappop(self.out)
        if len(self.out) >= self.fleet:
            return False
        heapq.heappush(self.out, back)
        return True


if __name__ == "__main__":
    log = ["a", "b", "a", "c", "a", "b"]
    assert top_k_users(log, 2) == ["a", "b"]
    assert top_k_users_sorted(log, 5) == ["a", "b", "c"]
    # grid paths: a 3 x 3 grid has 2 paths along the
    # bottom
    assert count_paths(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths(4, 5) == 9
    # an obstacle in the middle of the bottom row
    assert count_paths(3, 3, blocked=[(2, 1)]) == 1
    modulus = 10 ** 9 + 7
    assert count_paths(5, 30, mod=modulus) < modulus
    cars = [(1, 4), (2, 5), (4, 6), (5, 8)]
    assert min_cars(cars) == 2
    assert min_cars([]) == 0
    desk = RentalDesk(1)
    assert desk.rent(1, 4) and not desk.rent(2, 3)
    assert desk.rent(4, 6)
```

FULL QUESTION TEXT FROM THE SITE

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity.

Q1. Find top-K most active chat users

You are given a log of chat messages from a messaging application.

Input:

An array messages of length n , where each element messages[i] is the user_id (string or integer) of the sender of the i-th message.
An integer k (1 <= k <= number of distinct users).

Output:

Return a list of k user IDs corresponding to the users who have sent the most messages.

Details & requirements:

The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in messages).
If there are fewer than k distinct users, return all distinct users.
If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by user_id in ascending order.
Aim for an algorithm that is more efficient than sorting all users by count when k is much smaller than the number of distinct users.

Design and implement a function, for example:

List<UserId> topKActiveUsers(List<UserId> messages, int k)

Also explain:

At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
The time and space complexity of each approach.
Q2. Count paths in a 2D grid with constrained moves

You have a 2D grid with m rows and n columns. Index rows from top to bottom as 0..m-1 and columns from left to right as 0..n-1.

The start cell is the bottom-left corner: (m-1, 0) .
The end cell is the bottom-right corner: (m-1, n-1) .

From any cell (r, c), you may move only to the next column to the right, in one of up to three ways (if within bounds):

Right-up: (r-1, c+1)
Right: (r, c+1)
Right-down: (r+1, c+1)

You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

Given integers m and n , compute the total number of distinct paths from the start cell (m-1, 0) to the end cell (m-1, n-1) using only the allowed moves.

Requirements:

FULL QUESTION TEXT FROM THE SITE

Implement a function such as long countPaths(int m, int n) .
The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct.
Q3. Minimum cars needed for rentals and assignment

You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

An array of R rental records. Each record has:
rental_id : a unique identifier (e.g., integer or string).
pickup_time : an integer start time.
return_time : an integer end time, with pickup_time < return_time .

Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time t and another starts at time t, the same car can serve both.

Tasks:

Capacity : Compute the minimum number of cars required so that all rentals can be fulfilled without time conflicts on any single car.
Assignment : Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

An integer C = minimal number of cars needed.
A mapping from each rental_id to a car_id in the range 1..C , such that no two rentals assigned to the same car_id overlap in time.

Requirements & clarifications:

Overlap definition: two rentals with intervals [s1, e1) and [s2, e2) overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if e1 == s2 they do not overlap.
Aim for an algorithm that is efficient for up to, say, 10^5 rental records.
Discuss a naive approach and then an optimized approach (for example, using a min-heap/priority queue keyed by the earliest return time to track currently in-use cars).
Analyze the time and space complexity of your final solution.

STUDY PATH: question 149 of 167 (more practice)

Priority score 3.0 | Tries | asked 127 days ago | frequency Low

MEDIUM | 1Point3Acres | phone-screen | 2 of 5

Greedy Longest-Match Dictionary Tokenizer

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 215/228**

ABOUT THIS QUESTION

Topic: More practice - Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

APPROACH

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary key that is a prefix of the remaining text. Putting the keys in a trie lets you walk the text character by character from that position and remember the deepest end-of-key node you passed, instead of trying every length. If the tie-break rule changes (priority instead of length), store the priority at the terminal node and keep the best terminal seen.

Basis: Written for this question (longest-prefix matching with a trie, as in LeetCode 208 and the maximal-munch rule of lexers).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tokenize(text):
    out, i = [], 0
    while i < len(text):
        for size in range(max_key_len, 0, -1):
            key = text[i:i + size]
            if key in dictionary:
                out.append(dictionary[key])
                i += size
                break
        else:
            out.append(text[i])  # literal
            i += 1
    return out
```

TIME	SPACE
O(n * L)	**O(dictionary)**
lookups, each O(L)	

WHY IT IS SLOW

Every lookup hashes a substring, O(L^2) per position. Walking a trie of the keys one character at a time costs O(L) per position.

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

TIME	SPACE
O(n * L)	**O(total key length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The dictionary is huge.

A flat map of every key wastes memory because keys share prefixes. A compressed (radix) trie stores each shared stretch once, or shard the keys by first characters across machines. Lookup stays one pass over the text; only the node layout changes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Greedy Longest-Match Dictionary Tokenizer
#
# Phone-screen coding round (R1). Tokenize a text against
# a dictionary of key:id entries using greedy longest-
# match; emit the matched id when a key matches, otherwise
# output the unmatched character as a literal. One follow-
# up on changing the tie-breaking / priority rule for
# which token wins. A second follow-up appears when the
# dictionary is very large: discuss how to keep the
# matcher memory- and lookup-efficient at scale. Input: a
# text string and a dictionary mapping key (string) -> id.
# Sweep the text left-to-right. At each position, find the
# longest dictionary key that matches a prefix of the
# remaining text; emit its id. If no key matches at the
# current position, emit the single character as a literal
# and advance by one. Output the concatenation (or list)
# of ids and literal characters in order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The dictionary is huge.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary key
# that is a prefix of the remaining text. Putting the keys
# in a trie lets you walk the text character by character
# from that position and remember the deepest end-of-key
# node you passed, instead of trying every length. If the
# tie-break rule changes (priority instead of length),
# store the priority at the terminal node and keep the
# best terminal seen.
#
# BRUTE FORCE (say it first): Hash map lookup for every
# prefix length
# At every position try the prefix lengths from longest to
# shortest in a dictionary lookup. Correct and easy; each
# lookup hashes a substring, so it costs O(L^2) per
# position. Time O(n * L) lookups, each O(L). Space
# O(dictionary).


# BRUTE FORCE (say it first): Hash map lookup for every
# prefix length | time O(n * L) lookups, each O(L)
def tokenize_brute(text, table):
    out, i, longest = (
        [],
        0,
        max(map(len, table), default=0),
    )
    while i < len(text):
        for k in range(
            min(longest, len(text) - i), 0, -1
        ):
            if text[i : i + k] in table:
                out.append(table[text[i : i + k]])
                i += k
                break
        else:
            out.append(text[i])
            i += 1
    return out


# BASE QUESTION (optimal): Trie walk keeping the deepest
# (or best-priority) terminal
# Idea: Insert every key into a trie with its id at the
# last node. From each text position walk down while the
# next character exists, remembering the last terminal
# passed.
# FOLLOW-UP 1: The dictionary is huge.
# Answer: A flat map of every key wastes memory because
# keys share prefixes. A compressed (radix) trie stores
# each shared stretch once, or shard the keys by first
# characters across machines.
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id. priority(key): lower wins;
        # with
        # None the longest match wins
        self.priority = priority
        self.root = {}
        for key, ident in table.items():
            node = self.root
            for ch in key:
                node = node.setdefault(ch, {})
            # the end of a key stores the key and its id
            node["$"] = (key, ident)

    def better(self, new, old):
        # should the match `new` replace the match `old`?
        if old is None:
            return True
        if self.priority is None:
            # deeper (longer) matches come later: keep
            # them
            return True
        rank = self.priority
        return rank(new[0]) < rank(old[0])

    def longest_at(self, text, i):
        # walk the trie from position i
        node, best, j = self.root, None, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            end = node.get("$")
            if end and self.better(end, best):
                best = end
        return best

    def tokenize(self, text):
        out, i = [], 0
        while i < len(text):
            match = self.longest_at(text, i)
            if match is None:
                # no key starts here: keep the character
                out.append(text[i])
                i += 1
            else:
                key, ident = match
                out.append(ident)
                i += len(key)
        return out


# SIMPLER VERSION: one dictionary lookup per length
def tokenize_map(text, table):
    # simpler version: a dictionary lookup per length
    out, i = [], 0
    longest = max(map(len, table), default=0)
    while i < len(text):
        top = min(longest, len(text) - i)
        for size in range(top, 0, -1):
            if text[i:i + size] in table:
                out.append(table[text[i:i + size]])
                i += size
```

```
                break
        else:
            out.append(text[i])
            i += 1
    return out


if __name__ == "__main__":
    table = {"ab": 1, "abc": 2, "c": 3}
    t = Tokenizer(table)
    # abc is longer than ab, so it wins at position 0
    assert t.tokenize("abcab") == [2, 1]
    assert t.tokenize("xyz") == ["x", "y", "z"]
    assert t.tokenize("") == []
    assert tokenize_map("abcab", table) == [2, 1]
    # a single-letter key still applies the longest rule
    assert t.tokenize("cab") == [3, 1]
    # follow-up: priority by dictionary order instead
    order = list(table)
    pri = Tokenizer(table, priority=order.index)
    assert pri.tokenize("abc") == [1, 3]
```

NOTES FROM CANDIDATES

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.
Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.
Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).
If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

PREPARATION

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

STUDY PATH: question 150 of 167 (more practice)

Priority score 2.74 | Tries | asked 223 days ago

MEDIUM | PracHub | Technical Screen | 3 of 5

Check if all substrings are dictionary words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 43.5 | 30 DAYS 0 | PRIORITY 82/228

ABOUT THIS QUESTION

Topic: More practice - Tries

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: No direct match; related LC 139 Word Break

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-dictionary-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

APPROACH

Enumerate all substrings of length >= 3 and look each up in the set; exit on the first failure. Prune: if any length-3 window fails return false immediately. O(n^2) lookups.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the dictionary access (a set or a trie), the maximum length of s, and whether case matters. A string of length n has about n^2 / 2 substrings of length at least 3, and each is a separate requirement.

BRUTE FORCE TO SAY OUT LOUD

Enumerate every substring of length at least 3 and look it up. O(n^2) lookups, each costing O(length) to hash, so O(n^3) characters.

OPTIMAL IDEA

Exit on the first failure; checking all length-3 windows first prunes most inputs. To avoid the length factor, walk a trie: for each start i descend the trie along s[i:], and at every depth of at least 3 require the end-of-word flag; if the path breaks, return false. That is O(n^2) node steps in total.

DRY-RUN THESE CASES

A string shorter than 3 (true vacuously), cats (needs cat, ats, cats), a failure on the first window, a failure only on the longest substring.

LIKELY FOLLOW-UPS

Anagram version (next question), or return the first failing substring.

PITFALLS

Checking only the whole string. Treating a string shorter than 3 as false.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Every substring is a separate requirement, so no clever skipping exists: you must test about n^2 / 2 substrings. What you can improve is the cost per test and the early exit. A trie lets you extend a start position one character at a time without rehashing the whole substring, and the first failure ends everything.

Basis: Written; every contiguous substring of length >= 3 must be a dictionary word. The dictionary is a set (optionally preprocessed into a trie).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, dictionary):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            if s[i:j] not in dictionary:
                return False
    return True
```

TIME	SPACE
O(n^2)	**O(1)**
lookups, O(n^3) characters hashed	extra

WHY IT IS SLOW

About n^2 / 2 lookups and each hashes a whole substring, O(n^3) characters. Walking a trie from each start, one character at a time, is O(n^2) with no hashing of long strings.

OPTIMAL SOLUTION · Walk a trie from each start position

TIME	SPACE
O(n^2)	**O(dictionary size)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the first failing substring.

Same loops, but return the substring instead of False, which is what a user wants to see. Check the shorter ones first so the report is the smallest counterexample.

FOLLOW-UP 2 The string is very long.

The check is inherently quadratic in the number of substrings; any substring that fails ends the search, so cheap necessary checks first help: every length-3 window must be a word (O(n) lookups), then length 4, and so on.

FOLLOW-UP 3 The dictionary is huge and sharded.

Batch the lookups by length (a set per length) or use a Bloom filter in front of the real dictionary to reject most misses cheaply. **Cost:** still O(n^2) lookups, spread over shards (each lookup O(L)); a Bloom filter in front removes most remote calls.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check if all substrings are dictionary
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** is a valid dictionary word (i.e., for all i, j
# with 0 <= i <= j < n and j - i + 1 >= 3, s[i..j] in
# dict). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - You may assume
# dictionary lookups are available (e.g.,
# dict.contains(word)), or you can preprocess the
# dictionary. Example: - If s = "cats", you must check
# "cat", "ats", and "cats" (and any other substrings of
# length >= 3).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first failing substring.
# 2. The string is very long.
# 3. The dictionary is huge and sharded.
#
# INTERVIEW NOTES
# Every substring is a separate requirement, so no clever
# skipping exists: you must test about n^2 / 2 substrings.
# What you can improve is the cost per test and the early
# exit. A trie lets you extend a start position one
# character at a time without rehashing the whole
# substring, and the first failure ends everything.
#
# BRUTE FORCE (say it first): Enumerate each substring and
# look it up in a set
# Slice s[i:j] for every i, j with j - i >= 3 and test
# membership. Correct and simple; every lookup re-hashes
# the whole substring. Time O(n^2) lookups, O(n^3)
# characters hashed. Space O(1) extra.


# BRUTE FORCE (say it first): Enumerate each substring and
# look it up in a set | time O(n^2) lookups, O(n^3)
# characters hashed
def all_substrings_words_brute(s, words):
    return all(
        s[i:j] in words
        for i in range(len(s))
        for j in range(i + 3, len(s) + 1)
    )


# HELPERS for the base question
# FOLLOW-UP 1: Return the first failing substring.
# Answer: Same loops, but return the substring instead of
# False, which is what a user wants to see. Check the
# shorter ones first so the report is the smallest
# counterexample.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def first_failing(s, words, min_len=3):
    # the SHORTEST substring (length >= min_len) that is
    # not a
    # word, or None. Short lengths first gives the
    # smallest
    # counterexample, and cheap checks run before
    # expensive ones.
    words = set(words)
    longest = max(map(len, words), default=0)
    for length in range(min_len, len(s) + 1):
        # longer than every word: the first window already
        # fails
        if length > longest:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if piece not in words:
                return piece
    return None


# BASE QUESTION (optimal): Walk a trie from each start
# position
# Idea: For each start i descend the trie along s[i:]. At
# depth 1 and 2 only walk; from depth 3 on the current
# node must be marked as a word end.
def all_substrings_words(s, words, min_len=3):
    return first_failing(s, words, min_len) is None


# SAME CHECK with a trie
class Trie:
    """The same check with a trie."""

    def __init__(self, words):
        self.root = {}
        for w in words:
            node = self.root
            for ch in w:
                node = node.setdefault(ch, {})
            node["$"] = True

    def all_substrings(self, s, min_len=3):
        for i in range(len(s)):
            node = self.root
            for depth, ch in enumerate(s[i:], 1):
                node = node.get(ch)
                if node is None:
                    # no word has this prefix: fails if
                    # the
                    # substring is long enough to count
                    return len(s) - i < min_len
                if depth >= min_len and "$" not in node:
                    return False
        return True
```

```
if __name__ == "__main__":
    words = {"aaa", "aaaa", "aaaaa"}
    assert all_substrings_words("aaaaa", words)
    assert Trie(words).all_substrings("aaaaa")
    # "aab" is a window that is not a word
    assert first_failing("aaab", words) == "aab"
    assert first_failing("aab", words) == "aab"
    assert not all_substrings_words("aab", words)
    assert not Trie(words).all_substrings("aab")
    # shorter than 3 letters: nothing to check
    assert all_substrings_words("ab", set())
    assert Trie([]).all_substrings("ab")
    # longer than every word: fails immediately
    assert first_failing("aaaaaa", words) == "aaaaaa"
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, dictionary: list[str]) -> bool:
```

Sample call

```
solution('cat', ['cat'])
```

Test cases

Input	Expected
('cat', ['cat'])	True
('cats', ['cat', 'cas', 'cts', 'ats', 'cats'])	True

STUDY PATH: question 151 of 167 (more practice)

Priority score 2.44 | Tries | asked 225 days ago

MEDIUM | IN APP LC 208 | PracHub | Technical Screen | 4 of 5

Return dictionary words matching a prefix

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 106/228

ABOUT THIS QUESTION

Topic: More practice - Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/return-dictionary-words-matching-a-prefix

QUESTION

You are given a list of strings **`words`** (a dictionary) and a string **`prefix`**. Return all words in **`words`** that start with **`prefix`**. Clarifications/constraints:

- Matching is case-sensitive.
- If no words match, return an empty list.
- If **`words`** contains duplicates, include duplicates in the output.
- You may return the result in any order unless the interviewer asks for a specific ordering (e.g., lexicographic).

APPROACH

Sort once and binary search the prefix range, or build a trie for repeated queries. O(log n + m) per query after preprocessing.

HOW TO THINK ABOUT IT

If the dictionary is sorted, all words with a prefix form one contiguous block, and its edges are found with binary search: the first word >= prefix, and the first word that is >= prefix with its last character incremented. For many queries on a static list this beats a trie in memory.

Basis: Search Suggestions System (LeetCode 1268) and prefix queries on a sorted list.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words, prefix):
    found = [w for w in words
             if w.startswith(prefix)]
    return found
```

BRUTE FORCE

TIME	SPACE
O(W * L) per query	O(m)

WHY IT IS SLOW

Each query scans the whole dictionary, O(W * L). With the words sorted, binary search finds the block of matches directly.

OPTIMAL SOLUTION · Sort once, binary search the range

TIME	SPACE
O(log W + m) per query after O(W log W) sorting	O(W)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the first K matches (autocomplete).

Stop the forward walk after K words, as the limit parameter does; with the sorted list the first K in alphabetical order are already the first K in the block.

FOLLOW-UP 2 Words are added while queries run.

Keep the list sorted with bisect.insort (O(n) per insert, fast in practice) or use a balanced tree; a trie handles inserts in O(L).

FOLLOW-UP 3 Case-insensitive matching.

Store a casefolded sort key next to each word (a tuple (key, word)) and search on the key; do not lowercase the displayed word.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Return dictionary words matching a prefix
#
# You are given a list of strings words (a dictionary) and
# a string prefix. Return all words in words that start
# with prefix. Clarifications/constraints: - Matching is
# case-sensitive. - If no words match, return an empty
# list. - If words contains duplicates, include duplicates
# in the output. - You may return the result in any order
# unless the interviewer asks for a specific ordering
# (e.g., lexicographic).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the first K matches (autocomplete).
# 2. Words are added while queries run.
# 3. Case-insensitive matching.
#
# INTERVIEW NOTES
# If the dictionary is sorted, all words with a prefix
```

```
# form one contiguous block, and its edges are found with
# binary search: the first word >= prefix, and the first
# word that is >= prefix with its last character
# incremented. For many queries on a static list this
# beats a trie in memory.
#
# BRUTE FORCE (say it first): Filter the whole list per
# query
# Test startswith on every word. Correct and fine for one
# query; each query pays for the whole list. Time O(W * L)
# per query. Space O(m).


from bisect import bisect_left, bisect_right, insort


# BRUTE FORCE (say it first): Filter the whole list per
# query | time O(W * L) per query
def prefix_matches_brute(words, prefix):
    return sorted(
        w for w in words if w.startswith(prefix)
    )


# BASE QUESTION (optimal): Sort once, binary search the
# range
# Idea: With the words sorted, the matches are the words
```

```
# from bisect_left(prefix) until the first word that no
# longer starts with the prefix.
# FOLLOW-UP 1: Only the first K matches (autocomplete).
# Answer: Stop the forward walk after K words, as the
# limit parameter does; with the sorted list the first K
# in alphabetical order are already the first K in the
# block.
# FOLLOW-UP 2: Words are added while queries run.
# Answer: Keep the list sorted with bisect.insort (O(n)
# per insert, fast in practice) or use a balanced tree; a
# trie handles inserts in O(L).
class PrefixIndex:
    def __init__(self, words):
        # a sorted copy: all words with a prefix sit
        # together
        self.words = sorted(words)

    # FOLLOW-UP 1: limit= returns only the first K
    def matches(self, prefix, limit=None):
        # first possible position, then walk while it
        # matches
        lo = bisect_left(self.words, prefix)
        # one past the last word that starts with the
        # prefix
        hi = bisect_right(self.words, prefix + "")
        found = self.words[lo:hi]
        return found if limit is None else found[:limit]

    # FOLLOW-UP 2: words added while queries run
    def add(self, word):
        # words can arrive while queries run: keep it
        # sorted
        insort(self.words, word)


# SIMPLE SCAN (say it first)
def matches_scan(words, prefix):
    # the simple O(n * L) answer, in the original order
    return [w for w in words if w.startswith(prefix)]


# FOLLOW-UP 3: Case-insensitive matching.
# Answer: Store a casefolded sort key next to each word (a
# tuple (key, word)) and search on the key; do not
# lowercase the displayed word.
```

```
class CaseInsensitiveIndex:
    """Ignore case, but show the original words."""

    def __init__(self, words):
        keyed = ((w.casefold(), w) for w in words)
        self.items = sorted(keyed)

    def matches(self, prefix):
        key = prefix.casefold()
        lo = bisect_left(self.items, (key,))
        out = []
        while lo < len(self.items) and \
                self.items[lo][0].startswith(key):
            out.append(self.items[lo][1])
            lo += 1
        return out


if __name__ == "__main__":
    words = ["apple", "app", "apricot", "Apple", "banana",
             "app"]
    idx = PrefixIndex(words)
    # duplicates are kept; matching is case sensitive
    assert idx.matches("ap") == ["app", "app", "apple",
                                 "apricot"]
    assert idx.matches("x") == []
    assert idx.matches("") == sorted(words)
    assert idx.matches("ap", limit=2) == ["app", "app"]
    scan = sorted(matches_scan(words, "ap"))
    assert scan == idx.matches("ap")
    idx.add("apex")
    assert "apex" in idx.matches("ap")
    ci = CaseInsensitiveIndex(words)
    got = ci.matches("APP")
    assert got == ["app", "app", "Apple", "apple"]
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], prefix: str) -> list[str]:
```

Sample call

```
solution(['apple', 'app', 'banana', 'apply'], 'app')
```

Test cases

Input	Expected
(['apple', 'app', 'banana', 'apply'], 'app')	['apple', 'app', 'apply']
(['Cat', 'car', 'Cart', 'dog'], 'Ca')	['Cat', 'Cart']

STUDY PATH: question 152 of 167 (more practice)

Priority score 2.28 | Tries | asked 273 days ago

HARD | IN APP LC 648 | PracHub | Technical Screen | 5 of 5

Implement Longest-Match Text Replacement

POSTED 2026-01-20 | 273 DAYS AGO | OLDER

STUDY: TALK ALL TIME 31.9 30 DAYS 0 PRIORITY 120/228

ABOUT THIS QUESTION

Topic: More practice - Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #648 Replace Words

LeetCode match: LC 648 Replace Words / LC 833 Find And Replace in String

Posted on PracHub: 2026-01-20 (updated 2026-06-13)

Original: https://prachub.com/coding-questions/implement-longest-match-text-replacement

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed.

Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2",
 "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **`0`**, both **`"ab"`** and **`"abc"`** match, so choose the longest token **`"abc"`** and output **`"2"`**. Then **`"d"`** has no match, so output it literally. Then **`"xy"`** matches and outputs **`"3"`**. Finally, **`"z"`** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

QUESTION

1. Find every token that matches the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy":
    "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

Hints

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

APPROACH

Build a trie of tokens; scan the text and at each position walk the trie, remembering the longest terminal match; emit its id and jump ahead, else copy the char. Aho-Corasick if the text is large.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary token starting there. Checking every token costs the size of the dictionary per character, so share prefixes in a trie: walk the text from the current position down the trie, remember the last node that ends a token, and stop when the trie has no further edge. The remembered node is the longest match.

Basis: LeetCode 648 (Replace Words) and the longest-prefix matching used in tokenizers and routing tables. Scan left to right, at each position take the longest dictionary token that matches, else copy the character.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def replace(text, entries):
    out, i = [], 0
    while i < len(text):
        best = ""
        for token in entries:
            if text.startswith(token, i):
                if len(token) > len(best):
                    best = token
        if best:
            out.append(entries[best])
            i += len(best)
        else:
            out.append(text[i])
            i += 1
    return out
```

TIME	SPACE
O(len(text) * tokens * token length)	O(1) extra

WHY IT IS SLOW

Each position tests every token. A trie of the tokens is walked from i, remembering the last token end, so a position costs at most the longest token.

OPTIMAL SOLUTION · Trie walk that remembers the last token end

TIME	SPACE
O(len(text) * longest token)	O(total token length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The priority is not longest match: insertion order, a score, or a custom comparator.

Collect every token that matches at the position (walk the trie and gather all token ends along the path, not just the last) and choose with the comparator. The trie gives the candidates; the policy is separate.

FOLLOW-UP 2 The text is huge or the dictionary changes often.

Use Aho-Corasick if you need all matches in one pass over the text, or a trie that supports inserts and deletes; a longest-match scan is then linear in the text length times the longest token.

FOLLOW-UP 3 Overlapping matches or word boundaries.

Require a boundary before and after a token by checking the neighbouring characters when you record a candidate end, or tokenise on word boundaries first and look up whole words. **Cost:** still O(len(text) * longest token) with the trie walk; a boundary check is O(1) per candidate.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Longest-Match Text Replacement
#
# You are given a text string and a dictionary of
# replacement entries. Each dictionary entry has the form
# token:id, where token is a string pattern and id is the
# value that should be emitted when that token is matched.
# Implement a function that scans the text from left to
# right and produces an output sequence using greedy
# longest-prefix matching: 1. At the current text
# position, find all dictionary tokens that match the
# substring starting at that position. 2. If at least one
# token matches, choose the longest matching token, append
# its corresponding id to the output, and advance by the
# length of that token. 3. If no token matches, append the
# original character at the current position to the output
# and advance by one character. 4. Continue until the
# entire text has been processed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority is not longest match: insertion order, a
# score, or a custom comparator.
# 2. The text is huge or the dictionary changes often.
# 3. Overlapping matches or word boundaries.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary token
# starting there. Checking every token costs the size of
# the dictionary per character, so share prefixes in a
# trie: walk the text from the current position down the
# trie, remember the last node that ends a token, and stop
# when the trie has no further edge. The remembered node
# is the longest match.
#
# BRUTE FORCE (say it first): At every position try every
# dictionary token
```

```
# For each position test startswith for every token and
# keep the longest. Correct and the oracle; each position
# checks the entire dictionary. Time O(len(text) * tokens
# * token length). Space O(1) extra.


# BRUTE FORCE (say it first): At every position try every
# dictionary token | time O(len(text) * tokens * token
# length)
def replace_longest_brute(text, mapping):
    out, i = [], 0
    while i < len(text):
        best = None
        for tok in mapping:
            if text.startswith(tok, i) and (
                best is None or len(tok) > len(best)
            ):
                best = tok
        if best:
            out.append(mapping[best])
            i += len(best)
        else:
            out.append(text[i])
            i += 1
    return out


# BASE QUESTION (optimal): Trie walk that remembers the
# last token end
# Idea: Build a trie of the tokens (each token's last node
# stores its id). At position i walk down the trie along
# the text; whenever the current node stores an id, record
# it and its length.
# FOLLOW-UP 1: The priority is not longest match:
# insertion order, a score, or a custom comparator.
# Answer: Collect every token that matches at the position
# (walk the trie and gather all token ends along the path,
# not just the last) and choose with the comparator. The
# trie gives the candidates; the policy is separate.
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id. A trie shares the
        # prefixes.
        self.root = {}
        for token, ident in mapping.items():
            node = self.root
            for ch in token:
                node = node.setdefault(ch, {})
            node["$"] = (token, ident)

    def candidates(self, text, i):
        # every token that matches at position i, shortest
        # first (all ends along the trie path, not just
        # the last)
        out, node, j = [], self.root, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node:
                out.append(node["$"])
        return out

    # FOLLOW-UP 1: choose= picks by priority, not length
    def replace(self, text, choose=None,
                word_boundary=False):
        # choose(candidates) picks the winner; the default
        # is the
        # longest token
        choose = choose or (lambda cs: cs[-1])
        out, i = [], 0
        while i < len(text):
            found = self.candidates(text, i)
            # FOLLOW-UP 3: only whole-word matches
            if word_boundary:
                found = [c for c in found if
                         self.edge(text, i, len(c[0]))]
            if found:
                token, ident = choose(found)
                out.append(ident)
                i += len(token)
            else:
                # nothing matches here: keep the character
                out.append(text[i])
                i += 1
        return out

    @staticmethod
    def edge(text, i, size):
        # letters must not touch the match on either side
        before = i == 0 or not text[i - 1].isalnum()
        end = i + size
        after = end == len(text)
        after = after or not text[end].isalnum()
        return before and after


if __name__ == "__main__":
    table = {"ab": 1, "abc": 2, "c": 3}
    r = Replacer(table)
    # abc beats ab at position 0
    assert r.replace("abcab") == [2, 1]
    assert r.replace("xyz") == ["x", "y", "z"]
    assert r.replace("") == []
    # priority by insertion order instead of length
    order = list(table)
    first = r.replace("abc", lambda cs: min(
        cs, key=lambda c: order.index(c[0])))
    assert first == [1, 3]
    # whole words only
    words = Replacer({"cat": "C"})
    got = words.replace("cat concat cat", None, True)
    assert got[0] == "C" and got[-1] == "C"
    assert "C" not in got[1:-1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(text: str, dictionary: dict[str, str]) -> lis
t[str]:
```

Sample call

```
solution("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
```

Test cases

Input	Expected
("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})	["2", "d", "3", "z"]
("aaaaa", {"a": "X", "aa": "Y"})	["Y", "Y", "X"]

ALSO REPORTED · 1 more

Greedy Longest-Match Dictionary Tokenizer | 1Point3Acres | phone-screen | asked 2026-06-15 | Low

https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.

Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.

Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).

If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

Preparation: Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.

For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.

If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.